

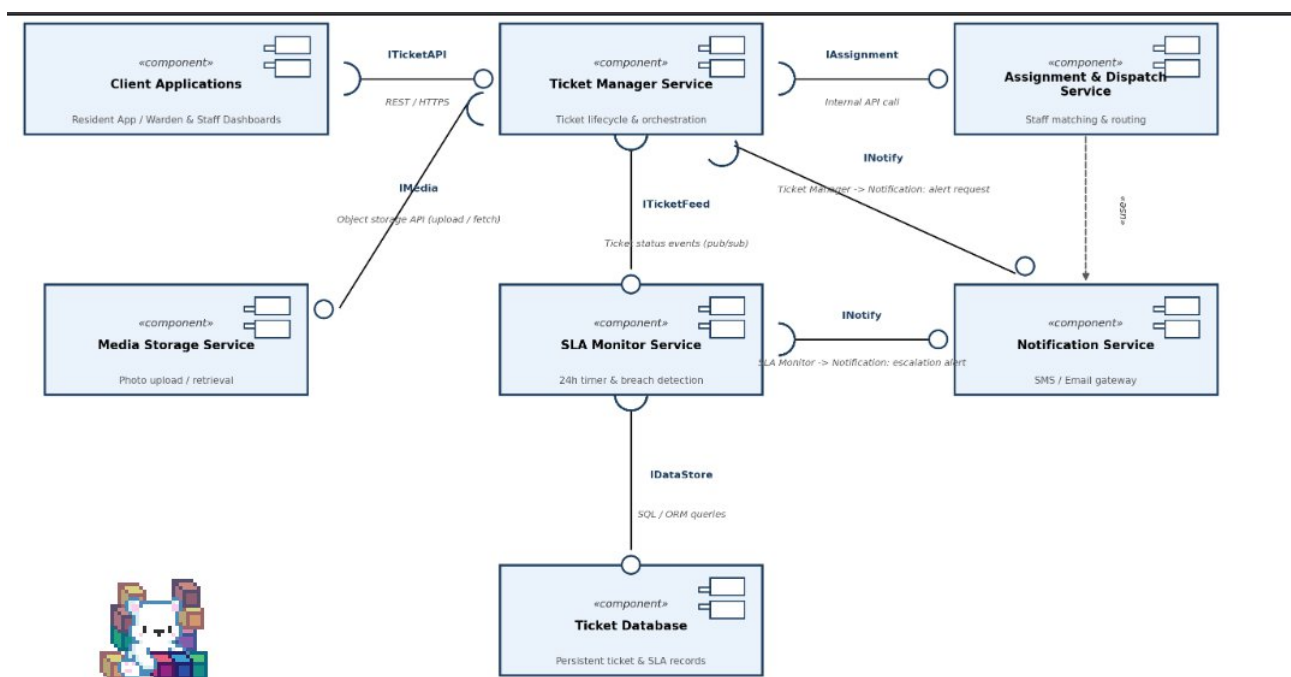
SE Lab 3: Architectural Style Justification

Hostel Maintenance & Issue Ticketing System: Problem 7

ROHINI VISHU

PES1UG24AM922

5H



Architecture Selection

We chose Microservices Architecture for the Hostel Maintenance & Issue Ticketing System.

Reason 1: Independent scaling around bursty, uneven load

Ticket creation and status checks happen steadily throughout the day, but SLA-breach escalations and SMS/Email notifications spike sharply after storms, power cuts, or plumbing failures that affect an entire block at once — exactly the kind of bursty external-I/O load described in the scenario's SLA and escalation requirement (NFR-001). A microservices split lets the Notification Service and SLA Monitor Service scale out independently during these spikes without over-provisioning the Ticket Manager or

Media Storage Service, which see comparatively flat traffic.

Reason 2: Fault isolation for a non-critical but high-blast-radius dependency

The Notification Service depends on third-party SMS/Email gateways that are outside the university's control and can time out or fail. In a microservices layout this dependency is isolated to one component: if the SMS/Email gateway is down, residents can still log tickets, wardens can still assign staff, and staff can still update ticket status — only the escalation alert is delayed. A tightly-coupled layered or single-server design would risk a slow external gateway call blocking the core ticket-logging path itself.

Security Advantage

Splitting the system into services lets each one enforce the least privilege it actually needs, which directly supports NFR-002 (role-based access and encrypted geo-tag data). The Media Storage Service, which holds photos of residents' private rooms, can be given its own access-controlled object store and encryption keys, separate from the Ticket Database credentials used by the Ticket Manager. A compromise of one service (e.g., the public-facing Notification Service) does not automatically expose ticket records or room photos, since each service only holds the credentials for its own interface.

Performance Benefit

Because the SLA Monitor Service runs as an independent component consuming a ticket-status event feed (ITicketFeed) rather than polling the shared Ticket Database directly, continuous 24-hour SLA timer checks do not add read load to the same database instance that serves resident-facing ticket submissions and warden dashboards. This keeps FR-004's real-time status view (updates reflected within 5 seconds) responsive even while thousands of SLA timers are being evaluated in the background.